7. Inheritance

A, B and C are classes

A is a super class. B is a sub class of A. C is a sub class of B.

Create three methods in each class, 2 methods are specific to each class and third method (override method) should be in all three Classes A, B and C

Create a class with main method. Create an object for each class A, B and C in main method and call every method of each class using its own object/instance.

Call an overridden method with super class reference to B and C class's objects

Runtime Polymorphism with Data Members/Instance variables, Repeat the above process only for data members

Solution:

```
class A {
    // Data member
    int value = 100;

    // Specific methods
    void methodA1() {
        System.out.println("Method A1 from Class A");
    }

    void methodA2() {
        System.out.println("Method A2 from Class A");
    }

    // Override method
    void display() {
        System.out.println("Display Method from Class A");
    }
}

class B extends A {
    // Data member
    int value = 200;

    // Specific methods
    void methodB1() {
        System.out.println("Method B1 from Class B");
    }

    void methodB2() {
        System.out.println("Method B2 from Class B");
    }

    // Override method
    @Override
    void display() {
        System.out.println("Display Method from Class B");
    }
}
```

```
class C extends B {
    // Data member
    int value = 300;

    // Specific methods
    void methodC1() {
        System.out.println("Method C1 from Class C");
    }

    void methodC2() {
        System.out.println("Method C2 from Class C");
    }

    // Override method
    @Override
    void display() {
        System.out.println("Display Method from Class C");
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {

        // Object of A
        A objA = new A();
        System.out.println("Class A Methods:");
        objA.methodA1();
        objA.methodA2();
        objA.display();

        // Object of B
        B objB = new B();
        System.out.println("\nClass B Methods:");
        objB.methodA1(); // inherited from A
        objB.methodA2(); // inherited from A
        objB.methodB1();
        objB.methodB2();
        objB.display();

        // Object of C
        C objC = new C();
        System.out.println("\nClass C Methods:");
        objC.methodA1(); // inherited from A
        objC.methodA2();
        objC.methodB1(); // inherited from B
        objC.methodB2();
        objC.methodC1();
        objC.methodC2();
        objC.display();

        // Runtime Polymorphism using superclass reference
        System.out.println("\nRuntime Polymorphism (Method Overriding):");
```

```
        A ref1 = new B();
        ref1.display();

        A ref2 = new C();
        ref2.display();

        // Runtime Polymorphism with Data Members
        System.out.println("\nRuntime Polymorphism with Data Members:");

        A dataRef1 = new B();
        System.out.println("A reference to B object value: " +
dataRef1.value);

        A dataRef2 = new C();
        System.out.println("A reference to C object value: " +
dataRef2.value);
    }
}
```

Explanation

- **A → B → C** is multilevel inheritance.
- display() method is **overridden** in all classes.
- **Runtime polymorphism** works for **methods**, so display() calls B and C versions.
- **Data members (instance variables) do not support runtime polymorphism** in Java. They follow **reference type**, so A ref = new B() prints A's variable value (100).